WEEK 2 ASSIGNMENT

Preparing Loan Data for Machine Learning

Fayz Liaqat | 16 October 2026

GitHub: github.com/fayzliaqat/week2-data-preparation

614 rows	13 original columns	2 target classes

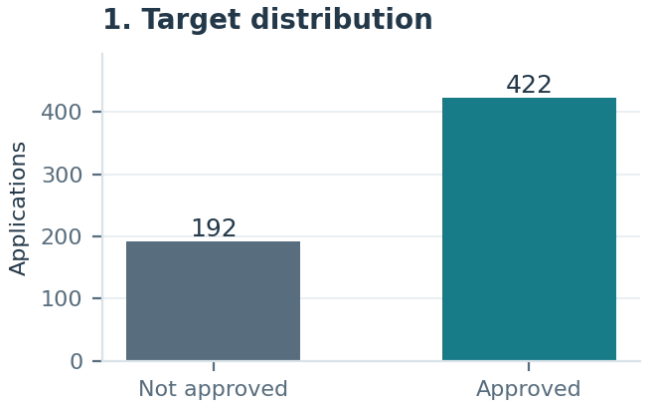
01 Dataset and objective

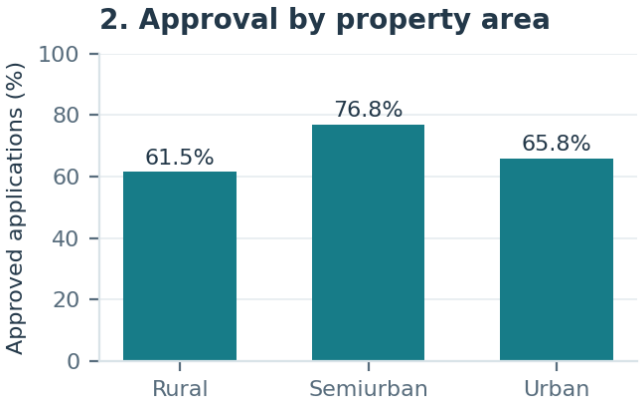
I chose **Loan Predication**, uploaded by amit parjapat on Kaggle. The downloaded CSV has numeric inputs, text categories, and missing values. Its target is **Loan_Status**, renamed to **loanStatus**: Y means approved and N means not approved. This is **supervised binary classification** because each application has a known class.

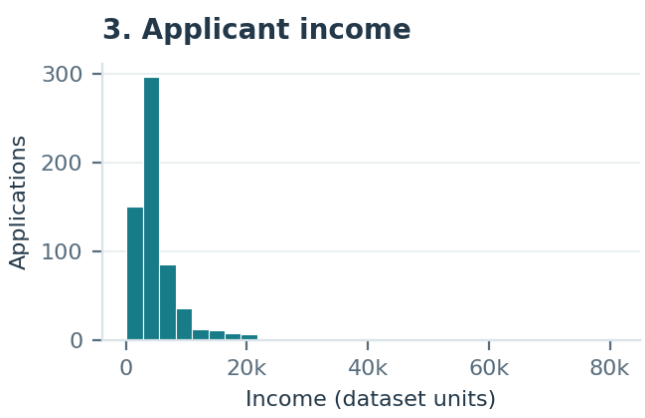
My aim was to produce X_train, X_test, y_train, and y_test with clean, encoded, and scaled inputs. I did not train a model.

02 EDA: what the data shows

I checked shape, info(), head(), and describe(), then plotted the target, categories, income, and numeric correlations in the notebook.







What I noticed

Class balance: 422 approved (68.73%) and 192 not approved. The classes are uneven.

Property area: Semiurban has the highest approval rate (76.8%) in this file.

Income: Most values are low compared with a few large values, so the distribution is right-skewed.

These are associations in this dataset, not evidence of cause.

CLEANING AND FEATURE ENGINEERING

03 Cleaning decisions

I found **0 exact duplicate rows** and still applied drop_duplicates(). There were **no missing target values**, so all 614 rows were retained. The notebook counts missing values in every column and shows a bar chart.

I kept useful incomplete columns and recorded the fill rules below. The actual fill values were learned later from training data only.

Column	Problem	Action	Reason
loanId	Unique ID	Drop	Identifies an application; no useful input pattern.
gender	13 missing	Training mode	Keep rows and fill with an existing category.
married	3 missing	Training mode	Fill a small number of gaps without losing rows.
dependents	15 missing	Training mode	Keep 3+ as a category, not an exact number.
selfEmployed	32 missing	Training mode	Keep the column using its most common training value.
loanAmount	22 missing	Training median	Less affected by large amounts than the mean.
loanTerm	14 missing	Training median	Simple numeric fill, following the assignment rule.
creditHistory	50 missing	Training median	Numeric rule; check that the learned median is 0 or 1.

The other columns had no missing values. Mode filling can make common categories more common. I kept large income values because a large number alone does not prove an error.

04 Two new features

Feature	How I made it	Why it may help
totalIncome	applicantIncome + coapplicantIncome	Represents the combined income available in an application.
hasCoapplicantIncome	1 if coapplicantIncome > 0; otherwise 0	Separates applications with an extra income contribution from those without one.

Both income columns were complete. These fixed row-by-row formulas use no target values or learned statistics. The flag does not prove whether a co-applicant exists, and I have not tested either feature's predictive benefit.

SPLIT, PREPROCESSING, AND FINAL CHECKS

05 Inputs, target, and the 80/20 split

I made **X** from the 13 input columns after dropping the ID and adding the two features. I made **y** from loanStatus, with N = 0 and Y = 1. The target is excluded from X.

The split uses **test_size=0.20, random_state=42, and stratify=y**: 491 training rows and 123 test rows. Stratification keeps similar class proportions.

Target class	Full dataset	Training set	Test set
0: not approved	192 (31.27%)	154 (31.36%)	38 (30.89%)
1: approved	422 (68.73%)	337 (68.64%)	85 (69.11%)

06 Preprocessing and avoiding data leakage

Numeric inputs: SimpleImputer fills gaps with training medians, then StandardScaler subtracts the training mean and divides by the training standard deviation. This puts the seven numeric features on comparable scales.

Text inputs: SimpleImputer fills gaps with training modes, then OneHotEncoder turns six categorical features into fifteen 0/1 columns without adding a false order. Unseen test categories are handled with handle_unknown='ignore'.

Train-only fitting: ColumnTransformer combines the pipelines. I used **fit_transform(XTrainRaw)** once and **transform(XTestRaw)** for testing. Imputer values, scaler statistics, and encoder categories were checked against training data. No preprocessing was fitted on test data.

The required full-data EDA is descriptive. Preprocessing rules were fixed in advance, and the two feature formulas learn no statistics. One-hot inputs remain 0/1; the target remains 0/1.

07 Final result

X_train	X_test	y_train	y_test
(491, 22)	(123, 22)	(491,)	(123,)

All four outputs have **zero missing values**. The input values are finite, feature columns match, and inputs align with their labels. Train/test indices do not overlap. The notebook ran top to bottom and exported all four CSV files.

08 Problems faced and what I learned

The main challenge was separating cleaning choices from learned fill values. I recorded the rules first, then learned the medians, modes, scaling values, and categories from training rows only.

The Kaggle description says 615 rows, but the downloaded CSV has 614 data rows plus its header. I learned to check the actual file, preserve categories such as 3+, and split before fitting preprocessing. Scaling does not remove outliers or fix skew. This report covers data preparation; no model accuracy is claimed.